



Security Audit

Mytho (Dapp)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	34
Conclusion	35
Our Methodology	36
Disclaimers	38
About Hashlock	39

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Mytho team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Mytho is a decentralized Web3 platform designed to empower communities by enabling them to create and manage their own digital tokens, known as "Totems." These Totems serve as symbols of shared identity, values, or interests, allowing groups to establish a collective treasury and governance structure. Users can mint new Totems or integrate existing tokens or NFTs to represent their community, facilitating a flexible and inclusive ecosystem. The platform's architecture supports the creation of layered content and a merit-based system, promoting active participation and contribution within communities. By leveraging blockchain technology, Mytho aims to foster decentralized collaboration and value creation among its users.

Project Name: Mytho

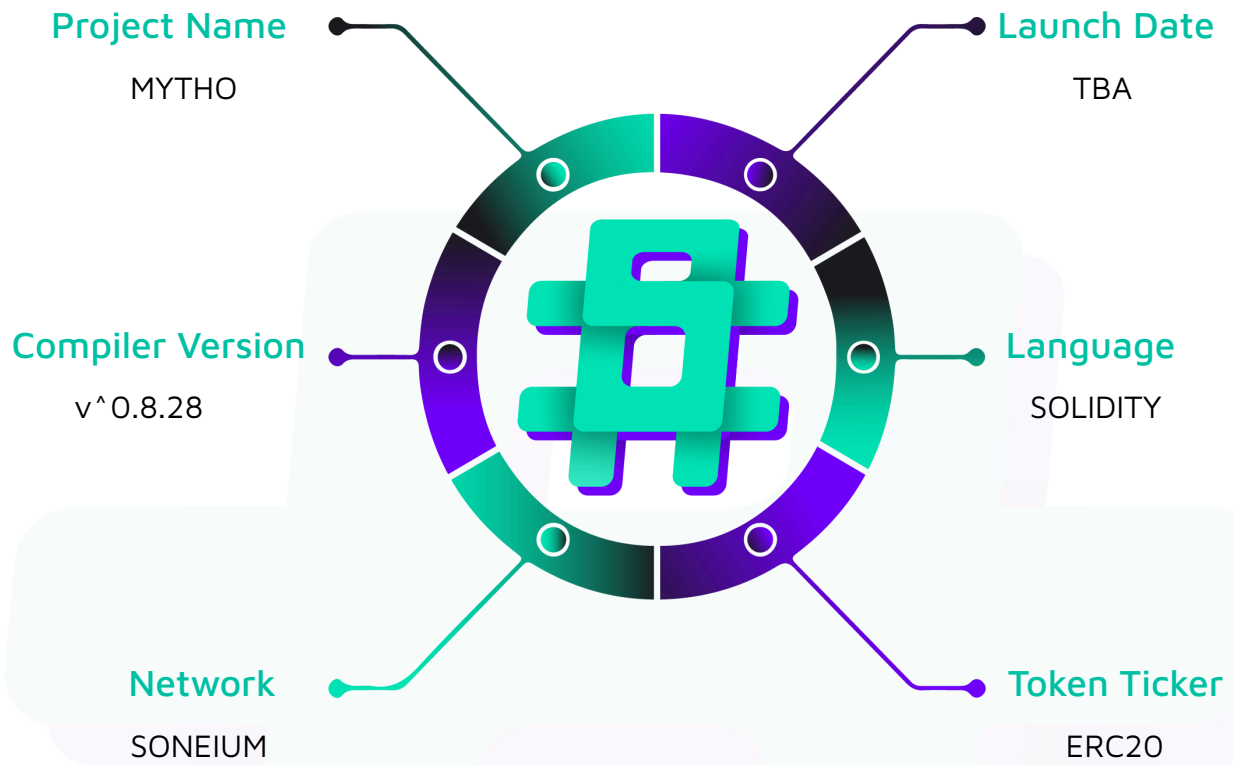
Project Type: dApp, Token.

Compiler Version: ^0.8.28

Website: <https://mytho.io/>

Logo:



Visualised Context:

Audit Scope

We at Hashlock audited the solidity code within the Mytho project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Mytho Smart Contracts
Platform	Soneium / Solidity
Audit Date	April, 2025
Contract 1	AddressRegistry.sol
Contract 1 MD5 Hash	8ac29046f494add73e9c33681dac2a01
Contract 2	BurnMintMYTHO.sol
Contract 2 MD5 Hash	3074b5a180bf092af626d03348f78d75
Contract 3	MeritManager.sol
Contract 3 MD5 Hash	084a9709906d9ddfca89231d738613dc
Contract 4	MYTHO.sol
Contract 4 MD5 Hash	14095eadba5d62e4bd5845cfc5cafd6d
Contract 5	Totem.sol
Contract 5 MD5 Hash	e6bc5d923c7a7748d005858c6d0f0126
Contract 6	TotemFactory.sol
Contract 6 MD5 Hash	206b3e9381a15176a0e27bebe5e958ed
Contract 7	TotemToken.sol
Contract 7 MD5 Hash	c5f11e64fae2eb9db666320ef6c01455
Contract 8	TotemTokenDistributor.sol
Contract 8 MD5 Hash	81a50e2796721821b48fa5df739b7363
Contract 9	Treasury.sol

Contract 9 MD5 Hash	56f1f0a19cd74e7546cc6c1a15ceb874
Contract 10	TokenHoldersOracle.sol
Contract 10 MD5 Hash	5776a10a23f9fb4d22be186fe30da1f8



Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

2 Gas Optimisations

4 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MYTHO.sol - Manage token distribution through vesting schedules and treasury allocations while controlling system-wide pauses.	Contract achieves this functionality.
MeritManager.sol Contract used to: - Register and track totems that participate in the Mytho ecosystem - Manage merit points earned by totems through community activity - Process monthly periods and convert merit points to MYTHO token rewards - Allow users to boost totems during special Mythum periods - Control blacklist status of problematic totems - Calculate and distribute MYTHO rewards based on proportional merit contribution - Track timing of reward periods and handle period transitions - Manage vesting allocations across 4 years of incentives distribution	Contract achieves this functionality.
TotemFactory.sol Contract used to: - Create standard totems with new token contracts or using whitelisted existing tokens - Maintain whitelist of approved external tokens for totem integration	Contract achieves this functionality.

- Collect creation fees and track all deployed totems - Deploy upgradeable totem proxies via beacon pattern - Register new totems with MeritManager for merit tracking	
TotemTokenDistributor.sol - Manage totem token sales and handle the entire lifecycle from initial sale to liquidity provision - Process totem token purchases and refunds during sale periods - Calculate token prices using Chainlink oracle feeds - Distribute collected payment tokens among treasury, creators, and AMM pools - Close sales periods automatically when thresholds are met - Create initial AMM liquidity with totem tokens and payment tokens - Register totems with MeritManager after successful sales	Contract achieves this functionality.
Totem.sol - Represent individual totem communities with their associated tokens and treasuries - Process token burning to distribute proportional shares of treasury assets - Manage the lifecycle from initial sale to post-sale token redemption - Support both standard and custom token implementations with different burning mechanics	Contract achieves this functionality.
TotemToken.sol	Contract achieves this

- Represent community-owned totem tokens with 1 billion initial supply - Restrict transfers during initial sale period for controlled distribution - Allow only the distributor to transfer tokens during sale period	functionality.
Treasury.sol - Stores and manages ERC20 tokens and native cryptocurrency - Restricts access to withdrawal functions with MANAGER role - Provides secure token transfers to specified addresses - Implements safety checks to prevent transfer errors	Contract achieves this functionality.
AddressRegistry.sol - Stores and manages addresses of key protocol contracts - Provides standardized access to core system components - Controls system-wide and totem-specific pause functionality	Contract achieves this functionality.
BurnMintMYTHO.sol - Enables controlled minting and burning by authorized addresses - Manages minting and burning permissions through access control lists - Supports cross-chain transfers via CCIP (Cross-Chain Interoperability Protocol) - Includes emergency pause mechanism to halt all transactions	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Mytho project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Mytho project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] MeritManager#initialize - Token Distribution Discrepancy

Description

There is a discrepancy between the contracts `MeritManager.sol` and `MYTHO.sol` regarding the annual token allocation for the Merit program. This discrepancy results in different amounts of tokens being distributed over the four-year distribution period, which can lead to unexpected economic consequences and potential problems with the functionality of the contract.

Vulnerability Details

The documentation states that 50% of the total token supply, which equals **1 billion** tokens, will be allocated for the incentive, thus 500 thousand million tokens will be allocated for the incentive over 4 years. The MYTHO contract correctly allocates tokens:

```
// Token distribution
uint256 public constant TOTAL_SUPPLY = 1_000_000_000 * 10 ** 18; // 1 billion tokens
with 18 decimals

// Totem incentives distribution (50% of total supply)
uint256 public constant MERIT_YEAR_1 = 200_000_000 * 10 ** 18; // 40% of incentives
uint256 public constant MERIT_YEAR_2 = 150_000_000 * 10 ** 18; // 30% of incentives
uint256 public constant MERIT_YEAR_3 = 100_000_000 * 10 ** 18; // 20% of incentives
uint256 public constant MERIT_YEAR_4 = 50_000_000 * 10 ** 18; // 10% of incentives
```

But the MeritManager contract already uses a different number of tokens:

```
function initialize(
    address _registryAddr,
    address[4] memory _vestingWallets
) public initializer {
    //...code
```

```

vestingWalletsAllocation = [
    175_000_000 ether,
    125_000_000 ether,
    100_000_000 ether,
    50_000_000 ether
];
//...code

```

As a result, the incentive drops from 500k to 450k. This number of tokens is used in the `_updateState` function, which calculates what number of tokens should be issued for a certain year and therefore they will not receive their 50 thousand tokens.

```

function _updateState() private {
    uint256 yearIdx = getYearIndex();

    // Check if we're still within the valid year range
    if (yearIdx >= 4) {
        return;
    }

    VestingWallet wallet = VestingWallet(payable(vestingWallets[yearIdx]));
    //...code

    if (_currentPeriod > lastProcessedPeriod) {
        uint256 slices = (30 days * 12) / periodDuration;

        // Process all completed periods up to but not including the current period
        for (
            uint256 period = lastProcessedPeriod;
            period < _currentPeriod;
            period++
        ) {
            => releasedMytho[period] =
            => vestingWalletsAllocation[yearIdx] /
                slices;

            emit MythoReleased(releasedMytho[period], period);
        }
    }
}

```

```
//...code
```

Impact

Unexpected Token Distribution: Users and totems would receive fewer tokens than documented in the first two years:

- Year 1: 25_000_000 fewer tokens (175M vs 200M)
- Year 2: 25_000_000 fewer tokens (125M vs 150M)

Community Trust Issues: Discrepancies between documented and actual token distribution could damage trust in the platform's transparency and governance.

Documentation mistake: The documentation also contains an incorrect calculation of the incentive amount of tokens. This mistake may affect the reputation of the protocol and confuse users

Supply and Distribution:

Total Supply: 1,000,000,000 MYTHO tokens.

Breakdown:

Totems Stimulation Program: 50% (500,000,000 tokens)

Distributed over four years to reward totem activity via Merit:

Year 1: 35% (175,000,000 tokens)

Year 2: 25% (125,000,000 tokens)

Year 3: 20% (100,000,000 tokens)

Year 4: 10% (50,000,000 tokens)

This starts big to attract users fast, then tapers off as totems stabilize.

Recommendation

Correct the values in the MeritManager contract and in the documentation itself to those specified in the MYTHO contract:

```
uint256 public constant MERIT_YEAR_1 = 200_000_000 * 10 ** 18; // 40% of incentives
uint256 public constant MERIT_YEAR_2 = 150_000_000 * 10 ** 18; // 30% of incentives
```

```
uint256 public constant MERIT_YEAR_3 = 100_000_000 * 10 ** 18; // 20% of incentives
uint256 public constant MERIT_YEAR_4 = 50_000_000 * 10 ** 18; // 10% of incentives
```

Status

Resolved

[M-02] TotemFactory#createTotemWithExistingToken - Duplicate Custom Token Registration in Totem Creation

Description

The TotemFactory contract allows you to create two different totems, but with the same custom totem token. This kind of feature can lead to several potential problems, namely scams and incorrect asset share calculation when burning totem tokens.

Vulnerability Details

The contract allows you to create totems using custom tokens that have the WHITELIST role. Once a token is approved and added to WHITELIST, anyone can create a totem with any token in WHITELIST.

```
function createTotemWithExistingToken(
    bytes memory _dataHash,
    address _tokenAddr,
    address[] memory _collaborators
) public whenNotPaused {
    if (_dataHash.length == 0) {
        revert InvalidTotemParameters("Empty dataHash");
    }

    // Collect fee in ASTR tokens
    _collectFee(msg.sender);

=>    if (!hasRole(WHITELISTED, _tokenAddr))
        revert NotWhitelisted(_tokenAddr);

    BeaconProxy proxy = new BeaconProxy(
        beaconAddr,
```

```

abi.encodeWithSignature(
    "initialize(address,bytes,address,bool,address,address[])",
    _tokenAddr,
    _dataHash,
    registryAddr,
    true,
    msg.sender,
    _collaborators
)
);

totemData[lastId++] = TotemData({
    creator: msg.sender,
=>    totemTokenAddr: _tokenAddr,
    totemAddr: address(proxy),
    dataHash: _dataHash,
    isCustomToken: true
});
// ...code
}

```

This kind of problem creates several vulnerabilities:

- 1) An attacker could exploit this vulnerability and create a totem with the newly created valid tokenA and pass off their totem as the original, thus luring users to their totem.
- 2) This totem token is used when calculating asset shares while burning their tokens in the burnTotemTokens function and in the getCirculatingSupply() function.

```

// Totem.sol
function burnTotemTokens(
    uint256 _totemTokenAmount
) external nonReentrant whenNotPaused {
    if (!isCustomToken && !salePeriodEnded) revert SalePeriodNotEnded();
    if (totemToken.balanceOf(msg.sender) < _totemTokenAmount)
        revert InsufficientTotemBalance();
    if (_totemTokenAmount == 0) revert ZeroAmount();

    // Check if there are MYTH0 tokens to claim in the current period
    uint256 currentPeriod = MeritManager(meritManagerAddr).currentPeriod();

```

```

uint256 pendingReward = MeritManager(meritManagerAddr).getPendingReward(
    address(this),
    currentPeriod
);
if (pendingReward > 0) {
    MeritManager(meritManagerAddr).claimMytho(currentPeriod);
}

// Get the circulating supply using the dedicated function
=> uint256 circulatingSupply = getCirculatingSupply();
if (circulatingSupply == 0) revert ZeroCirculatingSupply();

// Check balances of all token types
uint256 paymentTokenBalance = address(paymentToken) != address(0)
    ? paymentToken.balanceOf(address(this))
    : 0;
uint256 mythoBalance = mythoToken.balanceOf(address(this));
uint256 lpBalance = address(liquidityToken) != address(0)
    ? liquidityToken.balanceOf(address(this))
    : 0;

// Check if all balances are zero
if (paymentTokenBalance == 0 && mythoBalance == 0 && lpBalance == 0) {
    revert NothingToDistribute();
}

// Calculate user share for each token type based on circulating supply
=> uint256 paymentAmount = (paymentTokenBalance * _totemTokenAmount) /
    circulatingSupply;
=> uint256 mythoAmount = (mythoBalance * _totemTokenAmount) /
    circulatingSupply;
=> uint256 lpAmount = (lpBalance * _totemTokenAmount) / circulatingSupply;
// ...code
// Handle token disposal based on whether it's a custom token
if (isCustomToken) {
    // For custom tokens, transfer to treasuryAddr instead of burning
    totemToken.safeTransfer(treasuryAddr, _totemTokenAmount);
} else {
    // For standard TotemTokens, burn them

```

```

        totemToken.burn(_totemTokenAmount);
    }

    // Transfer the proportional payment tokens to the caller if there are any
    if (paymentAmount > 0) {
        paymentToken.safeTransfer(msg.sender, paymentAmount);
    }

    // Transfer MYTH0 tokens if there are any
    if (mythoAmount > 0) {
        mythoToken.safeTransfer(msg.sender, mythoAmount);
    }

    // Transfer LP tokens if there are any
    if (lpAmount > 0 && address(liquidityToken) != address(0)) {
        liquidityToken.safeTransfer(msg.sender, lpAmount);
    }
    // ...code
}

```

```

// Totem.sol
function getCirculatingSupply() public view returns (uint256) {
    uint256 totalSupply = totemToken.totalSupply();
    uint256 totemBalance = totemToken.balanceOf(address(this));
    uint256 treasuryBalance = isCustomToken
        ? totemToken.balanceOf(treasuryAddr)
        : 0;
    return totalSupply - totemBalance - treasuryBalance;
}

```

Pricing comes at the expense of the function `getCirculatingSupply`. The smaller it is, the more the user will get `paymentToken`, `mythoToken`, `liquidityToken`. For the function to return a smaller value, we need to wait for users from `totemA` to burn their tokens, thus increasing the treasury balance. The treasury address for all tokens is the same and hence the balance of this token will also be the same.

Impact

Duplication creates false circulating supply calculations

Recommendation

Add tracking of custom tokens used. But also consider preventing frontrunning while adding a custom token to WHITELIST, so that valid values are written to the TotemData structure: creator, dataHash and _collaborators

```
function createTotemWithExistingToken(
    bytes memory _dataHash,
    address _tokenAddr,
    address[] memory _collaborators
) public whenNotPaused {
    // ...code
    BeaconProxy proxy = new BeaconProxy(
        beaconAddr,
        abi.encodeWithSignature(
            "initialize(address,bytes,address,bool,address,address[])",
            _tokenAddr,
            _dataHash,
            registryAddr,
            true,
            msg.sender,
            _collaborators
        )
    );

    totemData[lastId++] = TotemData({
        creator: msg.sender,
        totemTokenAddr: _tokenAddr,
        totemAddr: address(proxy),
        dataHash: _dataHash,
        isCustomToken: true
    });
    // ...code
}
```

Status

Resolved

[M-03] TotemTokenDistributor#getPrice - Hardcoded Price Feed Values in Contract

Description

The TotemTokenDistributor contract contains a critical security issue where token prices default to a hardcoded value ($0.05 * 10^{18}$) when no price feed address is configured. This appears to be testing logic that was inadvertently left in production code, with proper error handling disabled. Despite the contract being upgradeable, this vulnerability presents significant risk until remediation is implemented.

```
function getPrice(address _tokenAddr) public view returns (uint256) {
    address priceFeedAddr = priceFeedAddresses[_tokenAddr];

    if (priceFeedAddr == address(0)) {
        // If no price feed is set for this token, return a default value. For test
purposes
        return 0.05 * 1e18;

        // revert NoPriceFeedSet(_tokenAddr);
    }
    // ..code
}
```

Vulnerability Details

The function returns a static value ($0.05 * 10^{18}$) when no price feed is configured. A proper error handling mechanism exists but is commented out. Comments explicitly indicate this is test code that requires removal.

Impact

1. Attackers can exploit the fixed price to arbitrage tokens, buying at below-market rates or selling at above-market rates.

2. Fixed pricing undermines fair market value determination.
3. Discovery of this vulnerability by users would undermine platform credibility.
4. Clear indication of insufficient code review processes.
5. Even with upgradeability, exploitation can occur until an upgrade is implemented.

Recommendation

Remove the fixed price return and uncomment the error revert in case the price is not set

Status

Resolved

Low

[L-01] MeritManager#isMythum - The Muthum period lasts over a week.

Description

As it says in the documentation the Muthum period is a special period in which the multiplication of all things takes place. And it is the last week of the month. The problem is that instead of 7 days, this period lasts 8 days. The `isMythum` function is responsible for this and due to incorrect calculations, it returns a duration of 8 days.

```
function isMythum() public view returns (bool) {
    uint256 currentPeriodNumber = currentPeriod();
    uint256 periodsAfterAccumulation = currentPeriodNumber -
        accumulatedPeriods;
    uint256 currentPeriodStart = startTime +
        (periodsAfterAccumulation * periodDuration);
    uint256 mythumStart = currentPeriodStart + ((periodDuration * 3) / 4);
    return block.timestamp >= mythumStart;
}
```

Proof of Concept

```
function test_MuthumPeriod() public {
    uint256 count;
    for (uint256 i = 0; i <= 40; i++) {
        vm.warp(i * 1 days);
        if (mm.isMythum()) count++;
    }
    assertEq(count, 8);
}
```

```
forge test --mt test_MuthumPeriod -vv
```

```
Ran 1 test for test/Test.t.sol:TestContract
```

```
[PASS] test_MuthumPeriod() (gas: 196169)
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 15.80ms (2.10ms CPU time)
```

```
Ran 1 test suite in 301.00ms (15.80ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

The number of days equal to 40 is taken conditionally to demonstrate that starting from 0 period up to 30 inclusive, the number of days for Muthum is 8 instead of 7. This number of days is not the beginning of Muthum in the second month.

Recommendation

Consider modifying this function to match the documentation.

Status

Resolved

[L-02] TotemTokenDistributor - Mismatch between comment and code

Description

This contract has a variable `slippagePercentage` which is used to calculate the minimum number of tokens that are added to the UniswapV2 pool. This variable is immediately assigned a value equal to 50, and judging by the comment, it will be 5%, so 1000 is 100%.

```
uint256 public slippagePercentage = 50; // Default 5% slippage (50/1000)
```

In the functions `_addLiquidity` and `setSlippagePercentage`, this variable is scaled with the constant `PRECISION` which is equal to 10_000. This inconsistency between the comment and the code is misleading for developers, auditors and users.

```
function _addLiquidity(
    address _totemTokenAddr,
    address _paymentTokenAddr,
    uint256 _totemTokenAmount,
    uint256 _paymentTokenAmount
) internal returns (uint256 liquidity, address liquidityToken) {
    // ...code
```

```

        uint256 minTotemAmount = (_totemTokenAmount *
            (PRECISION - slippagePercentage)) / PRECISION;
        uint256 minPaymentAmount = (_paymentTokenAmount *
            (PRECISION - slippagePercentage)) / PRECISION;
        // ...code
    }
    // ...
    /**
     * @notice Sets the slippage percentage for liquidity addition
     * @param _slippagePercentage New slippage percentage (multiplied by PRECISION)
     * @dev For example, 50 = 0.5%, 500 = 5%, 1000 = 10%
     */
    function setSlippagePercentage(
        uint256 _slippagePercentage
    ) external onlyRole(MANAGER) {
        if (_slippagePercentage >= PRECISION) revert InvalidShares();
        slippagePercentage = _slippagePercentage;
    }

```

The NatSpec comment for the `setSlippagePercentage` function shows an inconsistency in the initial comment.

There is also inconsistency between comment and code in such functions as `buy` and `sell`. Comments describe these functions so that you can buy and sell totems for a payment token, but in reality you can buy/sell a totem token.

```

/**
 * @notice Buy totems for allowed payment tokens
 * @param _totemTokenAddr Address of the totem token to buy
 * @param _totemTokenAmount Amount of totem tokens to buy
 */
function buy(
//...code
    IERC20(_totemTokenAddr).safeTransfer(msg.sender, _totemTokenAmount);
//...code
}
/**

```

```
* @notice Sell totems for used payment token in sale period
* @param _totemTokenAddr Address of the totem token to sell
* @param _totemTokenAmount Amount of totem tokens to sell
*/
function sell(
//...code
// send payment tokens and take totem tokens using SafeERC20
IERC20(_totemTokenAddr).safeTransferFrom(
    msg.sender,
    address(this),
    _totemTokenAmount
);
//...code
}
```

Recommendation

Fix the comments

Status

Resolved

Gas

[G-01] Contracts - Change the visibility of functions

Description

Public functions require more gas than external functions. Visibility of a public function is usually needed if it will be called externally and internally, i.e. the contract itself will call its function.

```
function pause() public onlyOwner {  
    _pause();  
}  
  
function unpause() public onlyOwner {  
    _unpause();  
}
```

Recommendation

Change the visibility of all functions that are not called by the same contract

Status

Resolved

[G-02] Contracts - There is no need to initialize variables with default values

Description

Most variables at the deployment stage and local variables in functions are assigned their default values, for uint it is 0, for address it is address(0), for bool it is false and so on.

Recommendation

You don't need to declare variables with their default values, this will save gas when deploying and when calling functions

```
//In the initialize function of the MeritManager contract
accumulatedPeriods = 0;

//In the initialize function of the Totem contract
salePeriodEnded = false;
```

Status

Resolved

QA

[QA-01] MYTHO - Unused Library Import

Description

The MYTHO contract contains dead code, importing and declaring usage of the EnumerableSet library that is never utilized within the contract:

```
using EnumerableSet for EnumerableSet.AddressSet
```

This presents several issues:

- Increased Deployment Costs
- Developer and auditor confusion

Recommendation

Remove dead code and unused imported library

Status

Resolved

[QA-02] MeritManager#getYearIndex- Incorrectly implemented function

Description

The _updateState function is used to distribute incentive tokens for 4 years, when this period passes it should just return, but it doesn't because of a misimplemented getYearIndex function. As you can see in the _updateState function if yearIdx >=4 it should return, but the getYearIndex function will always return 3 when exceeded.

```
function _updateState() private {
    uint256 yearIdx = getYearIndex();

    // Check if we're still within the valid year range
    if (yearIdx >= 4) {
        return;
    }
}
```

```

    }
    // ...code

```

```

function getYearIndex() public view returns (uint256) {
    uint256 slices = (30 days * 12) / periodDuration;
    return (currentPeriod() / slices) > 3 ? 3 : (currentPeriod() / slices);
}

```

Recommendation

Fix the `getYearIndex` function

Status

Resolved

[QA-03] Contracts - Missing Initialization of AccessControlUpgradeable

Description

It has been noted that the `AccessControlUpgradeable` contract, inherited in `TotemFactory` and `Treasury` is not initialized. Although both `__AccessControl_init()` and `__AccessControl_init_unchained()` functions are empty, so they do not have to be necessarily called, this could change in the future. Calling the initialization methods of all parent contracts is considered a good practice.

Not following good practices could leave any relevant contracts uninitialized, which could potentially lead to unexpected behavior or security vulnerabilities.

Recommendation

It is recommended to call the initialization methods of every parent contract.

Status

Acknowledged

[QA-04] MeritManager - TYPO

Description

The contract contains the `BLACKLIST` role and the contract also contains a `TotemInBlocklist` error related to this role. There is a typographical error in this error.

```
bytes32 public constant BLACKLISTED = keccak256("BLACKLISTED");  
error TotemInBlocklist();
```

Recommendation

Fix it to `TotemInBlacklist`.

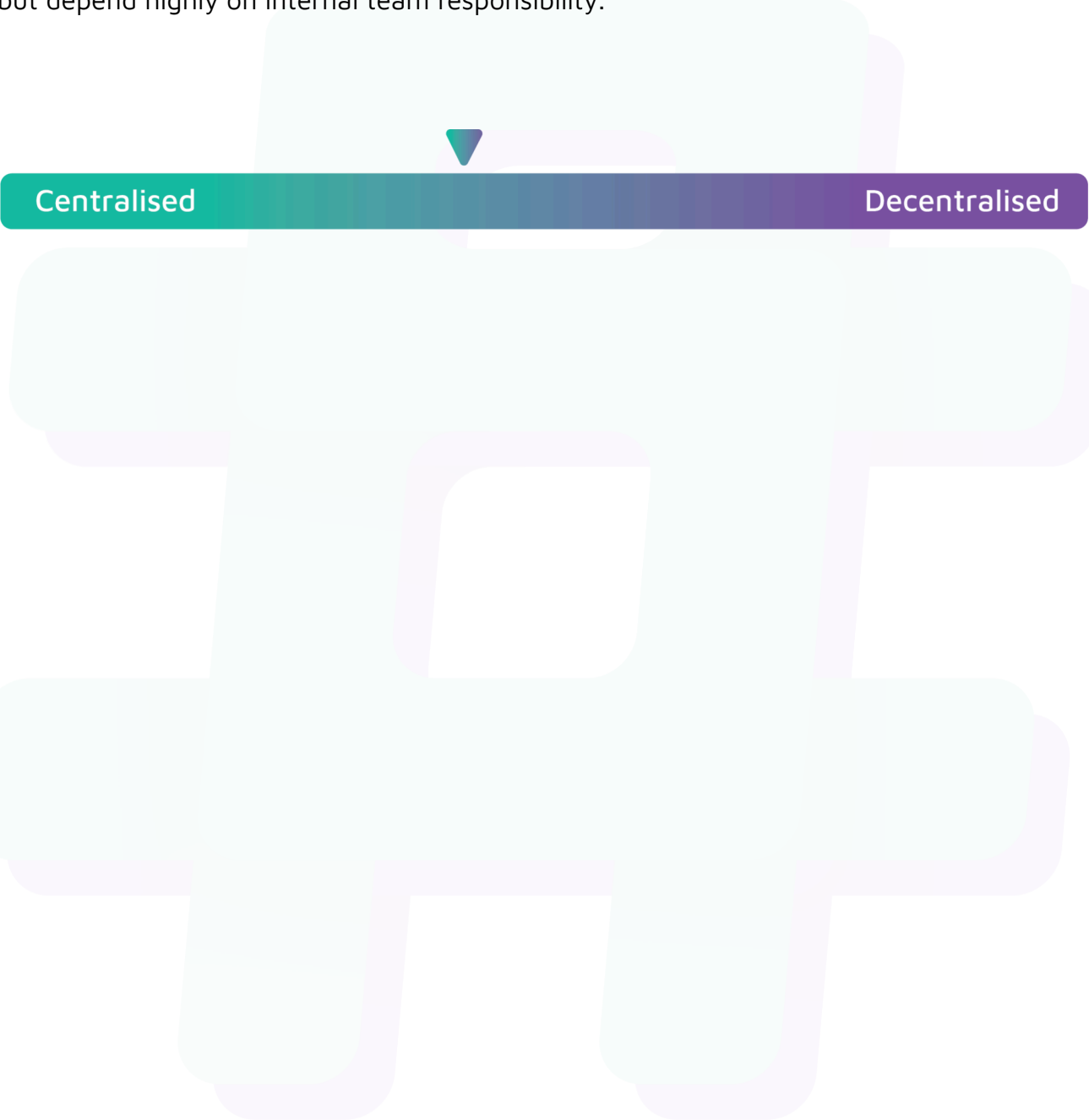
Status

Resolved

Centralisation

The Mytho project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Mytho project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.